Lecture Scribe Min-Cut Algorithm

L10_S1.pdf
PDF

Generate a lecture scribe suitable for exam preparation. Read and Analyze the document thoroughly and While writing the scribe, reason should be detailed and step by step through definitions, assumptions, proofs, and example solutions exactly as presented in the given slides. The Scribe should not miss any important detail as well as do not include anything which is not present in the slides. Also you can skip the '.ipynb' part at the ending of the slides.

Below is a **complete, exam-ready lecture scribe** strictly based on the uploaded slides of **Lecture 10: Randomized Min-Cut Algorithm**.
No additional material has been added beyond what appears in the slides.
(The `.ipynb` instruction at the end has been skipped as requested.)

Source:

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut algorithm is used in various applications related to:

- **Network connectivity**
- **Reliability**
- **Optimization**

Applications:

1. **Network Design**
    - Helps in improving efficiency of communication
    - Optimizes network flow
    - Used to find the **minimum capacity cut**
2. **Communication Networks**
    - Helps understand vulnerability of networks to failures
    - Useful in building robust and fault-tolerant networks
3. **VLSI Design**
    - Used for partitioning circuits into smaller components
    - Reduces interconnectivity complexity

Reference mentioned in slides:
Section 1.5, *Application: A Randomized Min-Cut Algorithm*, Probability and Computing (2nd Edition).

---

2. What is Min-Cut?

2.1 Cut-Set Definition

A **cut-set** in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

2.2 Minimum Cut Problem

Given:

$$G = (V, E)$$

where:

- V = set of vertices
- E = set of edges
- $n = |V|$

The **minimum cut (min-cut)** problem is:

> Find a minimum cardinality cut-set in G.

That is, find the smallest number of edges whose removal disconnects the graph.

2.3 Sensitivity of Min-Cut Algorithms

Min-cut algorithms such as **Karger's algorithm**:

- Are random.
- Are sensitive to initial choice of edges.
- If critical edges are contracted early, the algorithm might find a smaller cut.

---

3. Edge Contraction (Core Operation)

The **main operation** in the algorithm is:

Edge Contraction

Edge contraction:

> Removes an edge while simultaneously merging its two vertices.

If we contract edge (u, v):

Step-by-step:

1. Merge vertices u and v into a single vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges.

Important:

- The new graph may have **parallel edges**.
- The new graph has **no self-loops**.

(The diagrams on pages 15–16 visually demonstrate merging two vertices and removal of self-loops after contraction.)

---

4. Successful and Unsuccessful Runs

4.1 Successful Min-Cut Run

A successful run:

> Refers to success in the outcome of an algorithm designed to find the minimum cut in a graph.

The figure (page 19) shows:

- Sequence of contractions
- Final two supernodes
- Correct minimum cut obtained

4.2 Unsuccessful Min-Cut Run

An unsuccessful run:

> Refers to an iteration where the algorithm fails to correctly identify the minimum cut.

The figure (page 22) shows:

- Early contraction of critical edges
- Resulting cut is not minimum

Thus, randomness may lead to:

- Correct min-cut
- Or incorrect cut

---

5. Max-Flow Min-Cut Theorem

Statement

> In a flow network, the maximum amount of flow passing from source to sink is equal to the total weight of edges in a minimum cut.

Definitions

Capacity of a Cut

Sum of capacities of edges oriented from:

$$X \to Y$$

(where vertices in X connect to vertices in Y)

Minimum Cut

The cut with smallest possible capacity.

Minimum Cut Capacity

Capacity of the minimum cut.

Maximum Flow

Largest possible flow from:

$$S \to T$$

The diagram (page 26) illustrates:

- Source S
- Sink T
- Directed capacities
- Corresponding cut partition

---

6. Deterministic Min-Cut Algorithm

6.1 Stoer–Wagner Minimum Cut Algorithm

Let:

- s and t be two vertices of graph G
- $G/\{s, t\}$ be the graph obtained by merging s and t

Then:

A minimum cut of G is the smaller of:

1. A minimum s-t cut of G
2. A minimum cut of $G/\{s, t\}$

Why Theorem Holds?

Two cases:

1. There exists a minimum cut separating s and t
   → Then minimum s-t cut is minimum cut of G
2. No minimum cut separates s and t
   → Then minimum cut of $G/\{s, t\}$ is the answer

6.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

1. $A \leftarrow \{a\}$
2. While $A \neq V$:
   - Add to A the most tightly connected vertex
3. Return the cut weight as "cut-of-the-phase"

Algorithm 2: MinimumCut(G)

1. While $|V| \geqslant 1$:
   - Choose any $a \in V$
   - Call MinimumCutPhase(G, a)
   - If cut-of-the-phase < current minimum cut:
     - Store it
   - Shrink G by merging last two added vertices
2. Return minimum cut

Deterministic Properties

- Guarantees exact minimum cut
- May have higher time complexity for large graphs
- Time Complexity:

$$O(V \cdot E + V^2 \log V)$$

---

7. Randomized Min-Cut Algorithm

7.1 Why Randomized Algorithm?

Randomized algorithms:

- Provide probabilistic guarantee of success
- Provide more accurate estimate with fewer iterations

Advantages listed:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

---

8. Karger's Randomized Algorithm

The slide illustration (page 34) shows:

- Randomly pick an edge
- Remove it
- Fuse its endpoints
- Continue until two supernodes remain
- Output the cut between them

Example shown:

- Random edge choices produce non-minimum cut
- True minimum cut differs
- Demonstrates randomness effect

---

9. Recursive Randomized Min-Cut (Pseudocode)

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C

Step 1: Base Case

If:

$$n \leqslant 3$$

Then:

$$C \leftarrow \text{min-cut found using brute force}$$

Step 2: Recursive Case

Else:

For $i = 1$ to α:

1. Obtain $G^{'}$ by applying:

$$n - \frac{n}{\sqrt{\alpha}}$$

random contraction steps on G

2. $C^{'} \leftarrow$ Recursive-Randomized-Min-Cut($G^{'}, \alpha$)
3. If:
   - $i = 1$ OR
   - $|C^{'}| < |C|$

   Then:

$$C \leftarrow C^{'}$$

Final Step

Return C

---

10. Deterministic vs Randomized Min-Cut

Deterministic Min Cut

- Always guarantees exact minimum cut
- Higher time complexity for large graphs
- Stoer–Wagner:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min Cut

- Provides approximate minimum cut with high probability
- Karger's algorithm:

$$O(V^2)$$

Choice of Approach

> Any specific problem is responsible for deciding the approach.

---

11. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

End of Lecture

Thank You.

If you want, I can now convert this into:

- A clean LaTeX exam-ready document
- Short revision notes
- Or step-by-step probability derivation for the theorem (only from slides)

Generate a lecture scribe suitable for exam preparation. Read and Analyze the document thoroughly and While writing the scribe, reason should be detailed and step by step through definitions, assumptions, proofs, and example solutions exactly as presented in the given slides. The Scribe should not miss any important detail as well as do not include anything which is not present in the slides. Also you can skip the '.ipynb' part at the ending of the slides.

Your complete, structured, exam-ready lecture scribe has been generated in the canvas.

It strictly follows the slides:

- All definitions included
- All assumptions preserved
- Theorems written exactly as stated
- Pseudocode reproduced step-by-step
- No extra material added
- `.ipynb` section skipped as requested

If you'd like, I can now:

- Refine formatting for university submission
- Convert it into LaTeX
- Add structured theorem-proof formatting for exam writing practice
- Or compress it into a high-revision 2–3 page quick sheet

Tell me how you want to use it.

Generate a lecture scribe suitable for exam preparation. Read and Analyze the document thoroughly and While writing the scribe, reason should be detailed and step by step through definitions, assumptions, proofs, and example solutions exactly as presented in the given slides. The Scribe should not miss any important detail as well as do not include anything which is not present in the slides. Also you can skip the '.ipynb' part at the ending of the slides.

The lecture scribe has now been fully rewritten in a detailed, step-by-step, exam-oriented format strictly following the slides.

- All definitions preserved exactly
- All theorem statements written as given
- Case analysis structured clearly
- Pseudocode reproduced faithfully
- No additional material introduced
- `.ipynb` section excluded

If you want, I can now format it into a clean LaTeX submission version or convert it into a high-scoring 10–12 mark answer format for exams.

Generate a lecture scribe suitable for exam preparation. Read and Analyze the document thoroughly and While writing the scribe, reason should be detailed and step by step through definitions, assumptions, proofs, and example solutions exactly as presented in the given slides. The Scribe should not miss any important detail as well as do not include anything which is not present in the slides. Also you can skip the '.ipynb' part at the ending of the slides.

Cse400 Lecture10 Randomized Min Cut Scribe

CSE400 – Fundamentals of Probability in Computing

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut algorithm is used in various applications to solve problems related to:

• Network connectivity
• Reliability
• Optimization

Applications Mentioned in Slides

1. **Network Design**
   - Min cut helps in improving the efficiency of communication.
   - It helps in optimizing network flow.
   - The algorithm is used to find the minimum capacity cut.
2. **Communication Networks**
   - Minimum cut helps in understanding vulnerability of networks to failures.
   - It helps in building robust and fault-tolerant communication networks.
3. **VLSI Design**
   - Used for partitioning circuits into smaller components.
   - Leads to reduced interconnectivity complexity.

Reference (as mentioned in slides):
Section 1.5, Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition).

2. What is Min-Cut?

2.1 Definition: Cut-Set

A cut-set in a graph is:

A set of edges whose removal breaks the graph into two or more connected components.

Step-by-step interpretation:

1. Start with a connected graph.
2. Remove a specific set of edges.
3. If the graph becomes disconnected (two or more components), that set of edges is a cut-set.

2.2 Definition: Minimum Cut

Given a graph G = (V, E) with n vertices:

The minimum cut — or min-cut — problem is to find a minimum cardinality cut-set in G.

Step-by-step reasoning (as implied in slides):

1. Consider all possible cut-sets of G.
2. Determine the number of edges in each cut-set (cardinality).
3. Select the cut-set with the smallest cardinality.

That smallest cut-set is the minimum cut.

2.3 Sensitivity of Min-Cut Algorithms

Min-cut algorithms such as Karger's algorithm:

• Are random.
• Can be sensitive to the initial choice of edges.

If the algorithm contracts critical edges early in the process, it might find a smaller cut.

This explains why different executions of the algorithm may produce different results.

---

3. Edge Contraction (Main Operation)

The main operation in the algorithm is edge contraction.

Edge contraction is an operation that removes an edge from a graph while simultaneously merging the two vertices.

Contracting an Edge (u, v)

Step 1: Merge vertices u and v into a single vertex.
Step 2: Eliminate all edges connecting u and v.
Step 3: Retain all other edges in the graph.

Important properties after contraction:

• The new graph may contain parallel edges.
• The new graph has no self-loops.

(The slides visually demonstrate merging of vertices and removal of self-loops.)

---

4. Successful and Unsuccessful Min-Cut Runs

4.1 Successful Min-Cut Run

A successful min cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

From the slide illustration:

1. Random edges are selected.
2. Contractions are performed step by step.
3. The final two supernodes correspond to the correct minimum cut.

Thus, the algorithm correctly identifies the minimum cut in that execution.

4.2 Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

From the slide illustration:

1. Certain critical edges are contracted early.
2. The structure of the graph changes unfavorably.
3. The final cut obtained is not the true minimum cut.

This demonstrates the probabilistic nature of randomized contraction.

---

5. Max-Flow Min-Cut Theorem

5.1 Statement of Theorem

The max-flow min-cut theorem states:

“In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.”

5.2 Definitions Used in Theorem

1. Capacity of a cut
   = The sum of the capacity of the edges in the cut that are oriented from a vertex $\in X$ to a vertex $\in Y$.
2. Minimum cut
   = The cut in the network that has the smallest possible capacity.
3. Minimum cut capacity
   = The capacity of the minimum cut.
4. Maximum flow
   = The largest possible flow from source S to sink T.

(The slide diagram shows source S, sink T, directed capacities, and the cut partition.)

---

6. Deterministic Min-Cut Algorithm

6.1 Stoer–Wagner Minimum Cut Algorithm

Let s and t be two vertices of a graph G. Let G/{s, t} be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

1. A minimum s–t cut of G.
2. A minimum cut of G/{s, t}.

Justification (as stated in slides)

Case 1: There exists a minimum cut of G that separates s and t. Then a minimum s–t cut of G is a minimum cut of G.

Case 2: There is no minimum cut that separates s and t. Then a minimum cut of G/{s, t} gives the required minimum cut.

Thus, either of the two cases yields the global minimum cut.

6.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

```
A ← {a};
while A ≠ V do
add to A the most tightly connected vertex;
return the cut weight as the “cut of the phase”;
```

Algorithm 2: MinimumCut(G)

```
while |V| ≥ 1 do
choose any a from V;
MinimumCutPhase(G, a);
if the cut-of-the-phase is lighter than the current minimum cut then
store the cut-of-the-phase as the current minimum cut;
shrink G by merging the two vertices added last;
return the minimum cut;
```

6.3 Deterministic Properties

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.

Time complexity of Stoer-Wagner algorithm:

$O(V \cdot E + V^2 \log V)$,

where V is the number of vertices and E is the number of edges.

---

7. Randomized Min-Cut Algorithm

7.1 Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success. It provides a more accurate estimate of the minimum cut with fewer iterations.

Reasons listed in slides:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

---

8. Karger's Randomized Algorithm

Procedure (as illustrated in slides):

Step 1: Pick a random edge.
Step 2: Remove it.
Step 3: Fuse its two vertices into one.
Step 4: Repeat until only two supernodes remain.
Step 5: Output the cut defined by the remaining two supernodes.

The slide example demonstrates a run where random edges are picked such that the output cut is not minimal. The output cut differs from the actual minimum cut.

This example illustrates that the algorithm may fail in a single run.

---

9. Recursive Randomized Min-Cut

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input:
An undirected multigraph G with n vertices, and an integer constant $\alpha > 0$.

Output:
A cut C of G.

If $n \leqslant 3$ then
$C \leftarrow$ a min-cut of G found using brute force (exhaustive) search;
else
for $i \leftarrow 1$ to α do
$G' \leftarrow$ multigraph obtained by applying $n - n/\sqrt{\alpha}$ random contraction steps on G;
$C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α);
if $i = 1$ or $|C'| < |C|$ then
$C \leftarrow C'$;
return C;

---

10. Comparison: Deterministic vs Randomized Min-Cut

Any specific problem is responsible for deciding the approach.

Deterministic Min Cut

• Always guarantees an exact minimum cut.
• May have higher time complexity for large graphs.
• Time complexity: $O(V \cdot E + V^2 \log V)$.

Randomized Min Cut

• Provides an approximate minimum cut with high probability.
• Karger's algorithm has time complexity $O(V^2)$, where V is the number of vertices.

---

11. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$2 / (n(n - 1))$.

End of Lecture

The lecture scribe has now been fully refined again with:

- Clear step-by-step reasoning structure
- Explicit case breakdown in theorem section
- Proper logical sequencing for definitions
- Exact pseudocode as in slides
- No extra material added
- No missing slide content
- `.ipynb` portion excluded

If you'd like, I can now transform this into:

- A 15-mark exam answer format
- A short 5-page rapid revision version
- Or a perfectly formatted LaTeX submission draft

Just tell me your target use (exam, viva, assignment, etc.).

Generate a lecture scribe suitable for exam preparation. Read and Analyze the document thoroughly and While writing the scribe, reason should be detailed and step by step through definitions, assumptions, proofs, and example solutions exactly as presented in the given slides. The Scribe should not miss any important detail as well as do not include anything which is not present in the slides. Also you can skip the '.ipynb' part at the ending of the slides.

Below is the complete **exam-ready lecture scribe** for **Lecture 10: Randomized Min-Cut Algorithm**, written strictly from the provided slides.

- All definitions are included exactly as presented.

- All reasoning is written step-by-step.
- Theorems and justifications follow slide wording.
- Pseudocode is reproduced faithfully.
- No additional material beyond the slides is included.
- The `.ipynb` instruction section is skipped as requested.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut algorithm is used in various applications to solve problems related to:

- Network connectivity
- Reliability
- Optimization

Applications Mentioned in Slides

1. **Network Design**
   - Min cut helps in improving the efficiency of communication.
   - It helps in optimizing network flow.
   - The algorithm is used in network design to find the minimum capacity cut.
2. **Communication Networks**
   - For understanding the vulnerability of the networks to failures, minimum cut can be useful.
   - It helps building robust and fault-tolerant communication networks.
3. **VLSI Design**
   - In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components.
   - This leads to reduced interconnectivity complexity.

Reference mentioned in slides:
Section 1.5, *Application: A Randomized Min-Cut Algorithm* in *Probability and Computing*

(2nd Edition).

---

2. What is Min-Cut?

2.1 Definition: Cut-Set

A cut-set in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

Step-by-Step Interpretation

1. Start with a connected graph.
2. Remove a selected set of edges.
3. If the graph becomes disconnected (two or more components), that set of edges is a cut-set.

2.2 Definition: Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices,

The minimum cut — or min-cut — problem is:

> To find a minimum cardinality cut-set in G.

Step-by-Step Reasoning

1. Identify all possible cut-sets.
2. Compute the number of edges (cardinality) in each cut-set.
3. Select the cut-set with the smallest cardinality.

That smallest cut-set is called the **minimum cut**.

2.3 Sensitivity of Min-Cut Algorithms

Min-cut algorithms such as **Karger's algorithm**:

- Are random.
- Can be sensitive to the initial choice of edges.

If the algorithm happens to contract critical edges early in the process, it might find a smaller cut.

This explains why different runs of the algorithm may produce different results.

---

3. Edge Contraction (Main Operation)

The main operation in the algorithm is **edge contraction**.

Edge contraction is:

> An operation that removes an edge from a graph while simultaneously merging the two vertices.

Contracting an Edge (u, v)

When contracting edge (u, v):

Step 1: Merge vertices u and v into one vertex.
Step 2: Eliminate all edges connecting u and v.
Step 3: Retain all other edges in the graph.

Properties After Contraction

- The new graph may have parallel edges.
- The new graph has no self-loops.

(The slides visually demonstrate merging and removal of self-loops.)

---

4. Successful and Unsuccessful Min-Cut Runs

4.1 Successful Min-Cut Run

A successful min cut run refers to:

> The success in the outcome of an algorithm designed to find the minimum cut in a graph.

From the slide illustration:

1. Random edges are selected.
2. Contractions are performed step-by-step.
3. The final two supernodes correspond to the correct minimum cut.

Thus, the algorithm correctly identifies the minimum cut in that execution.

4.2 Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to:

> An iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

From the slide illustration:

1. Critical edges are contracted early.
2. The structure of the graph changes.
3. The final cut obtained is not the true minimum cut.

This demonstrates the probabilistic nature of the algorithm.

---

5. Max-Flow Min-Cut Theorem

5.1 Statement of Theorem

The max-flow min-cut theorem states:

> “In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.”

5.2 Definitions Used in Theorem

1. **Capacity of a cut**
   = The sum of the capacity of the edges in the cut that are oriented from a vertex ∈ X to a vertex ∈ Y.
2. **Minimum cut**
   = The cut in the network that has the smallest possible capacity.
3. **Minimum cut capacity**
   = The capacity of the minimum cut.
4. **Maximum flow**
   = The largest possible flow from source S to sink T.

(The slide includes a flow network diagram illustrating source S, sink T, capacities, and the cut partition.)

---

6. Deterministic Min-Cut Algorithm

6.1 Stoer–Wagner Minimum Cut Algorithm

Let s and t be two vertices of a graph G.
Let $G/\{s, t\}$ be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

1. A minimum s–t cut of G, and
2. A minimum cut of $G/\{s, t\}$.

Justification (As Stated in Slides)

Case 1:
There exists a minimum cut of G that separates s and t.
Then a minimum s–t cut of G is a minimum cut of G.

Case 2:
There is no minimum cut that separates s and t.
Then a minimum cut of $G/\{s, t\}$ gives the required minimum cut.

Thus, one of the two cases must produce the global minimum cut.

6.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

```
A ← {a};
while A ≠ V do
    add to A the most tightly connected vertex;
return the cut weight as the "cut of the phase";
```

Algorithm 2: MinimumCut(G)

```
while |V| ⩾ 1 do
    choose any a from V;
    MinimumCutPhase(G, a);
    if the cut-of-the-phase is lighter than the current minimum cut then
        store the cut-of-the-phase as the current minimum cut;
    shrink G by merging the two vertices added last;
return the minimum cut;
```

6.3 Deterministic Properties

- It always guarantees an exact minimum cut.
- It may have higher time complexity for large graphs.

Time complexity of Stoer-Wagner algorithm:

$$O(V \cdot E + V^2 \log V)$$

where V is the number of vertices and E is the number of edges.

---

7. Randomized Min-Cut Algorithm

7.1 Why Randomized Algorithm?

Randomized algorithms:

- Provide a probabilistic guarantee of success.
- Provide a more accurate estimate of the minimum cut with fewer iterations.

Reasons listed in slides:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

---

8. Karger's Randomized Algorithm

Procedure illustrated in slides:

Step 1: Pick a random edge.
Step 2: Remove it.
Step 3: Fuse its two vertices into one.
Step 4: Repeat until only two supernodes remain.
Step 5: Output the cut defined by the remaining two supernodes.

The example shown in slides demonstrates a run where random edges are picked in a way that the output cut is not minimal. The output cut differs from the actual minimum cut.

This illustrates the probabilistic behavior of the algorithm.

---

9. Recursive Randomized Min-Cut

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input:
An undirected multigraph G with n vertices, and an integer constant $\alpha > 0$.

Output:
A cut C of G.

If $n \leqslant 3$ then
  C ← a min-cut of G found using brute force (exhaustive) search;

```
else
  for i ← 1 to α do
    G′ ← multigraph obtained by applying n − n/√α random contraction steps on G;
    C′ ← Recursive-Randomized-Min-Cut(G′, α);
    if i = 1 or |C′| < |C| then
      C ← C′;
return C;
```

---

10. Comparison: Deterministic vs Randomized Min-Cut

Any specific problem is responsible for deciding the approach.

Deterministic Min Cut

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min Cut

- Provides an approximate minimum cut with high probability.
- Karger's algorithm has time complexity:

$$O(V^2)$$

where V is the number of vertices.

---

11. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

End of Lecture

Thank You.

Generate a lecture scribe suitable for exam preparation. Read and Analyze the document thoroughly and While writing the scribe, reason should be detailed and step by step through definitions, assumptions, proofs, and example solutions exactly as presented in the given slides. The Scribe should not miss any important detail as well as do not include anything which is not present in the slides. Also you can skip the '.ipynb' part at the ending of the slides.

Cse400 L10 Randomized Min Cut Final Scribe

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut algorithm is used in various applications to solve problems related to network connectivity, reliability, and optimization.

Applications

1. Network Design
    - Min cut helps in improving the efficiency of communication and optimizing network flow.
    - The algorithm is used in network design to find the minimum capacity cut.

2. Communication Networks
    - For understanding the vulnerability of the networks to failures minimum cut can be useful.
    - It helps building robust and fault-tolerant communication networks.
3. VLSI Design
    - In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components leading to reduced interconnectivity complexity.

Reference mentioned in slides:
Section 1.5. Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition).

---

2. What is Min-Cut?

2.1 Cut-Set

A cut-set in a graph is a set of edges whose removal breaks the graph into two or more connected components.

Step-by-step interpretation:

1. Start with a graph.
2. Remove a selected set of edges.
3. If the graph breaks into two or more connected components, that set of edges is a cut-set.

2.2 Minimum Cut

Given a graph G = (V, E) with n vertices, the minimum cut — or min-cut — problem is to find a minimum cardinality cut-set in G.

Step-by-step reasoning:

1. Consider all possible cut-sets.
2. Determine the number of edges in each cut-set (cardinality).
3. Select the cut-set with the smallest cardinality.

That smallest cut-set is the minimum cut.

2.3 Sensitivity of Min-Cut Algorithms

Min-cut algorithms, such as Karger's algorithm, are random and can be sensitive to the initial choice of edges.

If the algorithm happens to contract critical edges early in the process, it might find a smaller cut.

---

3. Edge Contraction

The main operation in the algorithm is edge contraction, which is operation that removes an edge from a graph while simultaneously merging the two vertices.

In contracting an edge (u, v):

1. Merge the two vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.

The new graph may have parallel edges but no self-loops.

---

4. Successful Min-Cut Run

A successful min cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

The slide figure shows a sequence of contractions that results in the correct minimum cut.

---

5. Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The slide figure shows contractions that lead to a cut which is not the minimum cut.

---

6. Max-Flow Min-Cut Theorem

Statement

The max-flow min-cut theorem states that:

“In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut”.

Definitions

Capacity of a cut = the sum of the capacity of the edges in the cut that are oriented from a vertex $\in$ X to a vertex $\in$ Y.

Minimum cut = the cut in the network that has the smallest possible capacity.

Minimum cut capacity = the capacity of the minimum cut.

Maximum flow = the largest possible flow from source S to sink T.

---

7. Deterministic Min-Cut Algorithm

Stoer–Wagner Minimum Cut Algorithm

Let s and t be two vertices of a graph G. Let G/{s, t} be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G
2. A minimum cut of G/{s, t}

Justification (as stated in slides)

The theorem holds since either there is a minimum cut of G that separates s and t, then a minimum s-t-cut of G is a minimum cut of G; or there is none, then a minimum cut of G/{s, t} does the job.

Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

A ← {a}; while A ≠ V do add to A the most tightly connected vertex; return the cut weight as the “cut of the phase”;

Algorithm 2: MinimumCut(G)

while |V| ⩾ 1 do choose any a from V; MinimumCutPhase(G, a); if the cut-of-the-phase is lighter than the current minimum cut then store the cut-of-the-phase as the current minimum cut; shrink G by merging the two vertices added last; return the minimum cut;

Deterministic Min-Cut Algorithm returns the minimum cut.

Time complexity: $O(V \cdot E + V^2 \log V)$, where V is the number of vertices and E is the number of edges.

---

8. Randomized Min-Cut Algorithm

Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success. It provides a more accurate estimate of the minimum cut with fewer iterations.

Advantages listed in slides:

• Efficiency
• Parallelization
• Approximation Guarantees
• Avoidance of Worst-Case Instances
• Heuristic Nature
• Robustness

Karger's Randomized Algorithm

Procedure:

1. Pick a random edge.
2. Remove it.
3. Fuse its endpoints into a single vertex.
4. Repeat until only two supernodes remain.
5. Output the cut defined by the remaining two supernodes.

The slide example demonstrates that random edge choices can produce a cut that is not the true minimum cut.

---

9. Recursive Randomized Min-Cut

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input: Undirected multigraph G with n vertices and integer constant $\alpha > 0$.

Output: A cut C.

If $n \leqslant 3$ then $C \leftarrow$ a min-cut of G found using brute force search; else for $i \leftarrow 1$ to α do $G' \leftarrow$ multigraph obtained by applying $n - n/\sqrt{\alpha}$ random contraction steps on G; $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α); if $i = 1$ or $|C'| < |C|$ then $C \leftarrow C'$; return C;

---

10. Comparison: Deterministic vs Randomized Min-Cut

Any specific problem is responsible for deciding the approach.

Deterministic Min Cut: • Always guarantees an exact minimum cut. • May have higher time complexity for large graphs. • Time complexity: $O(V \cdot E + V^2 \log V)$.

Randomized Min Cut: • Provides an approximate minimum cut with high probability. • Karger's algorithm has time complexity $O(V^2)$, where V is the number of vertices.

---

11. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least $2 / (n(n - 1))$.

End of Lecture

Your complete, strictly slide-based, exam-ready lecture scribe has now been freshly generated in the canvas.

It includes:

- All definitions exactly as in slides
- Step-by-step reasoning structure
- Theorem statement and justification exactly as written
- Full pseudocode reproduction
- Comparison section
- Probability theorem
- `.ipynb` portion excluded

If you'd like, I can now:

- Convert this into a perfectly formatted LaTeX submission
- Compress it into a 4–5 page rapid revision sheet
- Or restructure it into high-scoring exam-answer format (5, 10, 15 mark patterns)

Tell me your exam strategy and I'll tailor it precisely.